

Matplotlib — Questions & Answers

A study guide covering the basics of plotting with Matplotlib in Python.

1. What is Matplotlib?

Matplotlib is a Python library used for creating static, animated, and interactive visualizations such as graphs and charts. It is widely used in data analysis, scientific computing, and machine learning to visually represent data.

2. What is pyplot?

pyplot is a module within Matplotlib (`matplotlib.pyplot`) that provides a simple, MATLAB-like interface for creating plots. It offers easy-to-use functions such as `plot()`, `show()`, and `title()` to quickly generate and customize graphs.

```
import matplotlib.pyplot as plt
```

3. Difference between `plot()` and `scatter()`?

`plot()` is used to draw continuous lines connecting data points, making it ideal for showing trends over a continuous range (like time series data). `scatter()` plots individual data points without connecting them with lines, making it ideal for showing the relationship or distribution between two variables.

```
plt.plot(x, y) # connected line  
plt.scatter(x, y) # individual points, no line
```

4. What is a histogram?

A histogram is a chart that shows the frequency distribution of numerical data by dividing values into ranges called "bins" and displaying how many data points fall into each bin. It is created using `plt.hist()`.

```
plt.hist(data, bins=10)
```

5. What is a pie chart?

A pie chart is a circular chart divided into slices, where each slice represents a proportion of the whole. It is useful for showing percentage or relative composition of categories.

```
plt.pie(sizes, labels=labels)
```

6. What is a bar chart?

A bar chart represents categorical data using rectangular bars, where the length or height of each bar is proportional to the value it represents. It is useful for comparing quantities across different categories.

```
plt.bar(categories, values)
```

7. What is a subplot?

A subplot allows multiple plots to be displayed together in a single figure, arranged in a grid of rows and columns. This is useful for comparing several graphs side by side.

```
plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot  
plt.plot(x, y)
```

```
plt.subplot(1, 2, 2) # 2nd plot
plt.plot(x, y2)
```

8. What is savefig()?

savefig() is a function used to save the current figure to a file, such as a PNG, JPG, or PDF, instead of (or in addition to) displaying it on screen.

```
plt.savefig("myplot.png", dpi=300)
```

9. What is a legend?

A legend is a small box on a plot that explains what each line, color, or marker represents. It helps the viewer understand the different data series shown in a graph.

```
plt.plot(x, y, label="Sales")
plt.legend()
```

10. What is the purpose of grid()?

The grid() function adds gridlines to the background of a plot, making it easier to read exact values and compare data points across the chart.

```
plt.grid(True)
```

11. What is the use of xlabel() and ylabel()?

xlabel() and ylabel() are used to add descriptive labels to the horizontal (x) and vertical (y) axes of a plot, helping the viewer understand what each axis represents.

```
plt.xlabel("Time (s)")
plt.ylabel("Distance (m)")
```

12. How do you change line color and style?

Line color and style can be changed using the color (or shorthand like 'r' for red) and linestyle (or 'ls') parameters inside plot(). Common line styles include solid ('-'), dashed ('--'), dotted (':'), and dash-dot ('-').

```
plt.plot(x, y, color="red", linestyle="--")
plt.plot(x, y, "g:") # shorthand: green dotted line
```

13. What are markers?

Markers are symbols (such as circles, squares, or triangles) used to highlight individual data points on a plot. They are set using the marker parameter and are especially useful in scatter and line plots to make data points stand out.

```
plt.plot(x, y, marker="o") # circle markers
plt.plot(x, y, marker="*") # star markers
```

14. What is figure(figsize=...)?

figure(figsize=...) is used to create a new figure and control its size in inches (width, height). This is useful for making plots larger, smaller, or a specific proportion before adding data to them.

```
plt.figure(figsize=(8, 5)) # 8 inches wide, 5 inches tall
plt.plot(x, y)
```

15. How do you display multiple graphs?

Multiple graphs can be displayed either by using `subplot()/subplots()` to arrange several plots within one figure, or by calling `plt.figure()` multiple times to create separate figure windows, each shown together when `plt.show()` is called.

```
fig, axes = plt.subplots(2, 2) # 2x2 grid of plots
axes[0, 0].plot(x, y1)
axes[0, 1].plot(x, y2)
axes[1, 0].bar(cats, vals)
axes[1, 1].hist(data)
plt.show()
```